Q17: How Do Modern Models Vary the Transformer, and What Are the Alternatives?

DLS 2026 — Draft module

October 2026

Learning objectives

Students will decompose a transformer into design choices, relate those choices to context, memory, and throughput costs, distinguish sparse activation from sparse attention, and make a bounded comparison between transformer variants and state-space alternatives.

1 A family name is not a specification

"Transformer" names a broad architecture family rather than a single recipe. The original design alternates attention, feed-forward transformation, residual connections, and normalization [5]. A modern language model can retain this broad pattern while changing the attention heads, positional representation, normalization placement, activation, feed-forward width, expert routing, layer pattern, training objective, and inference cache policy. Consequently, two systems can both be called transformers while having very different training and serving behavior.

A useful analysis asks which component is being varied and which quantity is constrained. Is the goal lower training FLOPs, lower memory, faster decoding, longer usable context, greater parameter capacity, or a different inductive bias? One modification can improve one bottleneck while worsening another. A longer context window, for instance, may increase cache memory or leave retrieval reliability unresolved.

2 Attention and the decoding bottleneck

In autoregressive decoding, each new token needs access to prior key/value representations. Multi-query attention shares key/value heads across query heads, reducing key/value cache traffic; grouped-query attention is an intermediate compromise. The original multi-query proposal motivates the systems issue directly: memory bandwidth and cache size can limit generation even when arithmetic throughput is available [3]. These variants change the parameterization and cache layout, not the basic idea of content-dependent attention.

Positional representation is another variation. Rotary position embedding rotates query and key features according to position, encoding relative position information in the attention calculation [4]. Context-extension methods may adapt positional handling, training mixtures, and attention patterns. A claim that a system supports a very long window should therefore be tested with evidence placed at different distances, distractors, and paraphrases rather than inferred from a configuration number.

3 Dense layers, sparse experts, and sparse attention

Mixture-of-experts (MoE) architectures route a token to one or a few feed-forward experts. Switch Transformer uses sparse expert activation to scale capacity while activating a limited path per token [1]. This is distinct from sparse attention, which limits which token positions attend to which others. It is also distinct from quantization, which changes numerical representation, and from batching, which changes serving execution.

The distinction matters for honest cost claims. Total parameters affect storage; active experts affect per-token computation; routing and inter-device communication affect throughput; attention/cache design affects context cost. There is no single "model size" number that settles every comparison.

4 Alternatives: state-space and recurrent sequence models

State-space sequence models update a learned state as they scan a sequence, offering a different route to linear-time sequence processing. Mamba introduces selective state spaces whose behavior depends on the input [2]. This can be attractive when dense attention's quadratic interaction pattern is costly. It does not follow that every task requiring selective access to a distant detail is solved equally well by a compressed state.

The right comparison is empirical and task-specific. Match token budget, parameter/active-compute budget, data, precision, hardware, and decoding policy. Then evaluate ordinary accuracy, long-range retrieval, compositional transfer, latency, memory, and failure modes. A result on one sequence benchmark does not establish a universal replacement for transformers at frontier scale.

5 Worked design diagnosis

Imagine a deployment that must answer questions about a 100,000-token technical document. If memory is the bottleneck, grouped-query attention, quantization, or retrieval may help. If the answer requires comparing many distant passages, a state-space recurrence may be efficient but needs direct evaluation of selective recall. If serving cost is dominant, MoE may lower active computation but add routing and communication costs. The lesson is not to name one winner; it is to translate a product requirement into measurable architectural hypotheses.

6 Takeaway

Modern transformers vary along several independent axes. Attention-head/cache design addresses decoding memory; positional treatment addresses sequence location; MoE addresses conditional feed-forward capacity; and alternatives such as state-space models offer a different sequence mechanism. Compare them under a declared workload and resource budget rather than treating labels as verdicts.

References

[1] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23, 2022.

[2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[3] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[4] Jianlin Su et al. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[5] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Companion videos: decoder-only transformers

A curated progression — *visual intuition* → *decoder architecture* → *working implementation.* All links verified live.

1. **3Blue1Brown — Transformers, the tech behind LLMs** (27:14). Visual intuition: tokens to embeddings, through transformer layers, to next-token probabilities; strong on embedding geometry, the output projection, and softmax with temperature. https://www.youtube.com/watch?v=wjZofJX0v4M

2. **StatQuest (Josh Starmer) — Decoder-Only Transformers, Clearly Explained** (36:44). Patient, step-by-step treatment of the decoder-only architecture used for autoregressive generation. https://www.youtube.com/watch?v=bQ5BoolX9Ag

3. **Andrej Karpathy — Let's build GPT: from scratch, in code, spelled out** (1:56:20). Turns understanding into implementation: a working GPT-style model in PyTorch, architecture connected to code and training (assumes Python/tensors/PyTorch). https://www.youtube.com/watch?v=kCc8FmEb1nY

For a technical audience, Karpathy is the single strongest recommendation; for an undergraduate sequence, the three together form a useful progression.